

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 5

дисциплина: Архитектура компьютера —

Студент: Ромеро Гаспар Фернандо Алексей

Группа: НКАбд-02-25

МОСКВА

2025 г.

Оглавление

Цель заботы

Теоретическое введение

2.1 Midnight Commander (mc)

2.2 Структура программы на NASM

Выполнения лабораторной работы

3.1 Работа в Midnight Commander и создание первой программы

3.2. Подключение внешнего файла in_out.asm

Выполнение самостоятельной работы

4.1 Вывод введенной строки (без in_out.asm)

4.2 Вывод введенной строки (с in_out.asm)

Выводы

Список литературы

Цель работы

Приобретение практических навыков работы в Midnight Commander. Освоение инструкций языка ассемблера `mov` и `int`.

Теоретическое введение

2. 1 Midnight Commander (mc)

Midnight Commander (mc) — это кроссплатформенный файловый менеджер с открытым исходным кодом для Unix-подобных систем, работающий в текстовом режиме. Согласно официальной документации, это «программа для просмотра каталогов и управления файлами, работающая в терминале» (Midnight Commander Wiki, 2023). Его двухпанельный интерфейс позволяет эффективно управлять файлами без графического интерфейса.

Основные функции

Инструмент предоставляет полные возможности управления файлами с помощью клавиатуры. Как отмечает Linux Documentation Project (2022), «MC позволяет копировать, перемещать, удалять и просматривать файлы как на локальных, так и на удалённых системах». Его система меню на функциональных клавишах (F1–F10) обеспечивает быстрый доступ к часто используемым операциям.

Интеграция с системой

Согласно документации Red Hat (2023), «Midnight Commander тесно интегрируется со средой командной оболочки (shell), позволяя переключаться между визуальным интерфейсом и командной строкой с помощью Ctrl+O». Эта интеграция делает его ценным инструментом для системных администраторов, часто работающих в консоли.

Преимущества в профессиональных средах

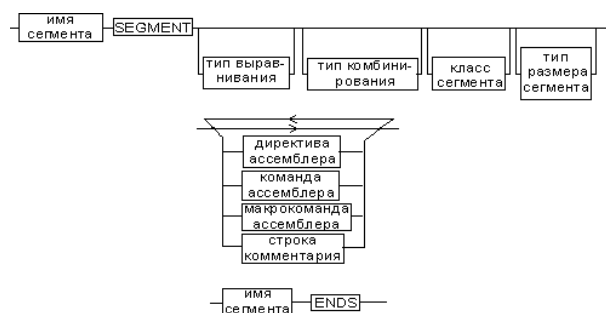
OpenSource.com (2022) подчёркивает, что «на серверах без графического интерфейса или при подключении по SSH MC предоставляет интуитивно понятный способ навигации и управления файлами». Его поддержка протоколов, таких как FTP, SFTP и SMB, расширяет полезность при удалённом администрировании.

КЛАВИША	ОСНОВНАЯ ФУНКЦИЯ	ТЕХНИЧЕСКОЕ ОПИСАНИЕ
F1	Помощь	Показывает контекстную документацию и горячие клавиши
F2	Пользовательское меню	Доступ к пользовательским меню и скриптам
F3	Просмотр файла	Показывает содержимое файлов во внутреннем просмотрщике
F4	Редактировать файл	Открывает файл в настроенном редакторе
F5	Копировать	Копирует файлы/каталоги из активной панели в неактивную
F6	Переместить/Переименовать	Перемещает элементы или меняет имена
F7	Создать каталог (Mkdir)	Создает новый каталог в активной панели

F8	Удалить	Удаляет выбранные файлы или каталоги
F9	Главное меню	Активирует верхнюю строку меню
F10	Выход	Закрывает Midnight Commander
TAB	Переключить панель	Переключает между левой и правой панелями
INSERT	Выделить	Отмечает/снимает отметку с текущего файла
CTRL+\	Корень системы	Переходит в корневой каталог (/)
CTRL+~	Домашний каталог	Переходит в домашний каталог пользователя
CTRL+O	Показать оболочку	Показывает/скрывает подлежащий терминал
ALT+?	Найти файл	Начинает поиск файлов по имени
CTRL+R	Обновить	Обновляет содержимое активной панели
ПРОБЕЛ	Быстрый просмотр	Быстрый предварительный просмотр выбранного файла
CTRL+X + C	Сравнить файлы	Сравнивает файлы между панелями
CTRL+X + D	Сравнить каталоги	Сравнивает содержимое каталогов
ALT+TAB	Изменить режим панели	Переключает между различными видами панели

2.2 Структура программы на NASM

NASM (Netwide Assembler) — это модульный ассемблер с открытым исходным кодом, специально разработанный для архитектур x86 и x86-64. Его структура организована по принципу многоэтапной обработки, которая отдельно управляет фазами анализа, трансляции и генерации кода. Внутренний поток выполнения NASM включает три основных этапа: лексико-синтаксический анализ, в ходе которого исходный код преобразуется в токены; макроподстановку, обрабатывающую директивы `%define` и повторяющиеся структуры; и генерацию кода, переводящую мнемоники в двоичные опкоды. Ассемблер реализует иерархическую таблицу символов для управления локальной и глобальной областями видимости, а также движок выражений, вычисляющий константы во время ассемблирования.



2.3. Основные инструкции и системные вызовы

Базовые инструкции составляют фундаментальный набор операций, которые процессор может выполнять напрямую. К ним относятся операции передачи данных (MOV, PUSH, POP), арифметические и логические операции (ADD, SUB, AND, OR), управление потоком выполнения (JMP, CALL, RET) и операции сравнения (CMP, TEST). Каждая инструкция работает с регистрами процессора, адресами памяти или непосредственными значениями в соответствии с архитектурой x86/x86-64. Синтаксис операндов следует порядку назначение, источник для инструкций с двумя операндами.

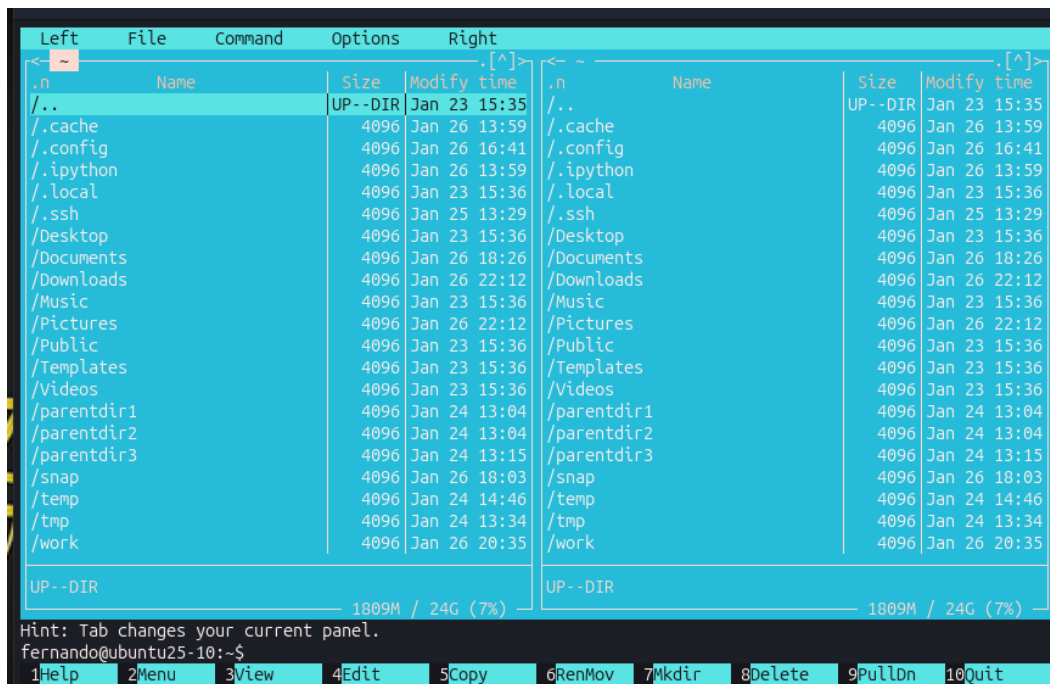
Системные вызовы (system calls) представляют собой интерфейс между программой в пользовательском пространстве и ядром операционной системы. В архитектурах x86-64 они вызываются с помощью инструкции SYSCALL, тогда как в 32-битных x86 используется INT 0x80. Каждый системный вызов имеет уникальный номер (например, 60 для exit, 1 для write) и специфические параметры, передаваемые через регистры: RAX содержит номер системного вызова, RDI — первый параметр, RSI — второй, RDX — третий, R10 — четвёртый, R8 — пятый и R9 — шестой. Правильное управление этими соглашениями о вызовах необходимо для взаимодействия с ядром.

Выполнения лабораторной работы

3.1 Работа в Midnight Commander и создание первой программы

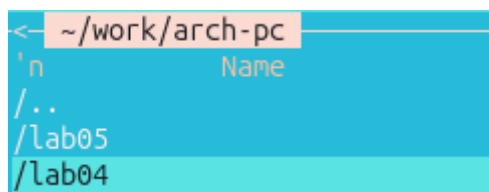
Вам необходимо запустить Midnight Commander. Создайте рабочую директорию с именем lab05, затем создайте файл с именем lab5-1.asm, отредактируйте его, добавив программу из Листинга 5.1, после чего скомпилируйте и запустите её.

Откройте терминал и введите команду ``mc`` для запуска Midnight Commander

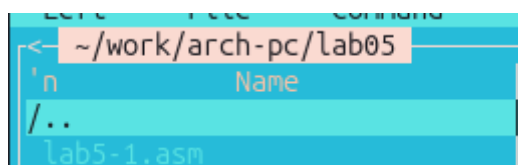


С
помощью
панели
навигации
перейдите в
каталог
~/work/arch-pc,
созданный в
предыдущей
лабораторной

работе. Нажмите F7, чтобы создать новый каталог, и введите имя lab05.



Перейдите в созданную директорию lab05, введите `touch lab5-1.asm` в командной строке под панелями и нажмите Enter. Файл появится на панели.



Далее выберите файл lab 5-1.asm и нажмите F4, чтобы открыть его во встроенном редакторе.

```
GNU nano 8.4 /home/fernando/work/arch-pc/lab05/lab5-1.asm
;---
; Программа вывода сообщения на экран и ввода строки с клавиатуры
;---
;--- Объявление переменных ---
SECTION .data          ; Секция инициализированных данных
msg: DB 'Введите строку:',10 ; сообщение плюс символ перевода строки
msgLen: EQU $-msg      ; Длина переменной 'msg'
SECTION .bss           ; Секция не инициализированных данных
buf1: RESB 80          ; Буфер размером 80 байт
;--- Текст программы ---
SECTION .text          ; Код программы
GLOBAL start          ; Начало программы
start:                ; Точка входа в программу
;--- Системный вызов write
mov eax,4             ; Системный вызов для записи (sys_write)
mov ebx,1             ; Описатель файла 1 - стандартный вывод
mov ecx,msg           ; Адрес строки 'msg' в 'ecx'
mov edx,msgLen        ; Размер строки 'msg' в 'edx'
int 80h              ; Вызов ядра
;--- Системный вызов read ---
mov eax,3             ; Системный вызов для чтения (sys_read)
mov ebx,0             ; Дескриптор файла 0 - стандартный ввод
mov ecx,buf1          ; Адрес буфера под вводимую строку
mov edx,80            ; Длина вводимой строки
int 80h              ; Вызов ядра
;--- Системный вызов exit ---
[ Read 29 lines ]
^O Help      ^O Write Out  ^F Where Is   ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File  ^L Replace    ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```

Сохраните файл (F2) и выйдите из редактора (F10). Нажатие клавиши F3 на файле lab5-1.asm подтверждает сохранение его содержимого.

```
/home/fernando/work/arch-pc/lab05/lab5-1.asm [----] 62 L:[ 1+15 16/ 30] *(1026/1890b) 0010 0x00A [*][X]
;---
; Программа вывода сообщения на экран и ввода строки с клавиатуры
;---
;--- Объявление переменных ---
SECTION .data          ; Секция инициализированных данных
msg: DB 'Введите строку:',10 ; сообщение плюс символ перевода строки
msgLen: EQU $-msg      ; Длина переменной 'msg'
SECTION .bss           ; Секция не инициализированных данных
buf1: RESB 80          ; Буфер размером 80 байт
;--- Текст программы ---
SECTION .text          ; Код программы
GLOBAL start          ; Начало программы
start:                ; Точка входа в программу
;--- Системный вызов write
mov eax,4             ; Системный вызов для записи (sys_write)
mov ebx,1             ; Описатель файла 1 - стандартный вывод
mov ecx,msg           ; Адрес строки 'msg' в 'ecx'
mov edx,msgLen        ; Размер строки 'msg' в 'edx'
int 80h              ; Вызов ядра
;--- Системный вызов read ---
mov eax,3             ; Системный вызов для чтения (sys_read)
mov ebx,0             ; Дескриптор файла 0 - стандартный ввод
mov ecx,buf1          ; Адрес буфера под вводимую строку
mov edx,80            ; Длина вводимой строки
int 80h              ; Вызов ядра
;--- Системный вызов exit ---
mov eax,1             ; Системный вызов для выхода (sys_exit)
mov ebx,0             ; Выход с кодом возврата 0 (без ошибок)
int 80h              ; Вызов ядра
```

Скройте панели mc с помощью Ctrl + O, чтобы получить доступ к командной строке, затем переведите, свяжите и запустите программу.

```
fernando@ubuntu25-10:~/work/arch-pc/lab05$ nasm -f elf lab5-1.asm
fernando@ubuntu25-10:~/work/arch-pc/lab05$ ld -m elf_i386 -o lab5-1 lab5-1.o
ld: warning: cannot find entry symbol _start; defaulting to 00049000
fernando@ubuntu25-10:~/work/arch-pc/lab05$ ./lab5-1
Введите строку:
Fernando Alexei
```

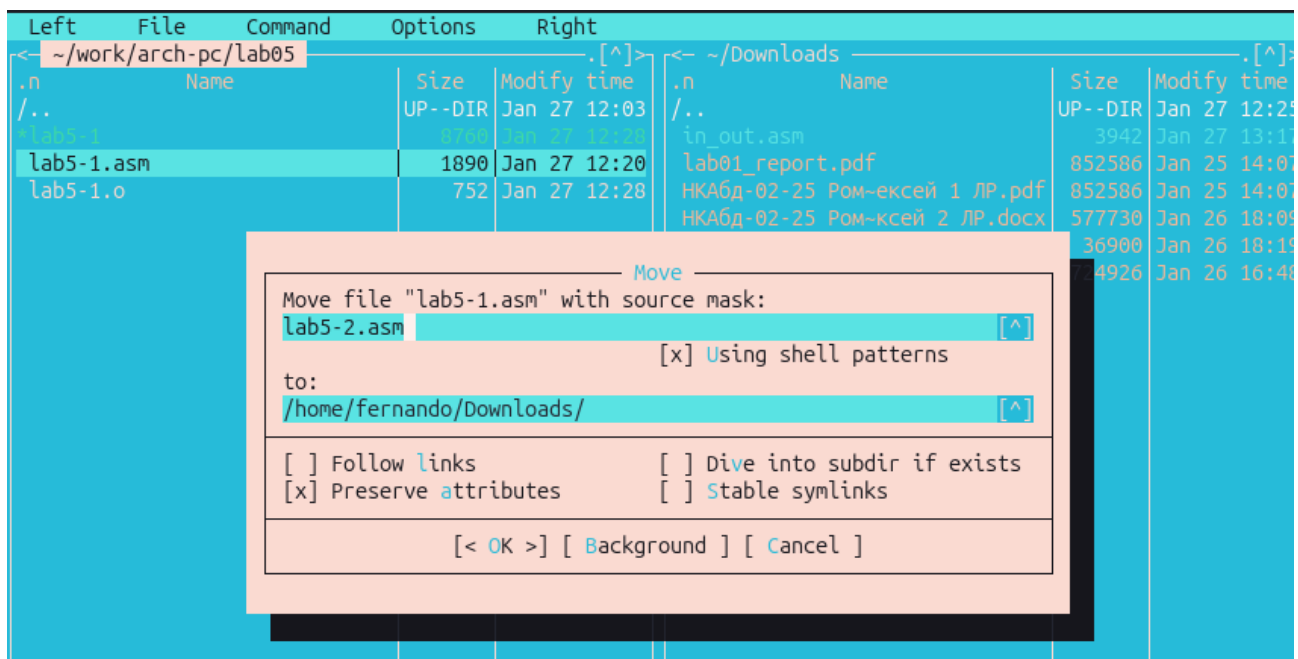
3.2 Подключение внешнего файла in_out.asm

Вам необходимо загрузить файл in_out.asm, скопировать его в свою рабочую директорию, создать копию файла lab5-1.asm под названием lab5-2.asm и изменить её, чтобы она использовала процедуры из файла in_out.asm.

В одной панели МС откройте свою рабочую директорию ~/work/arch-pc/lab05, а в другой панели МС (используя клавишу Tab) откройте директорию ~/Downloads. Выберите файл in_out.asm и нажмите F5 (Копировать), подтвердив копирование в lab05.

Left	File	Command	Options	Right			
<-	~/work/arch-pc/lab05			<- ~/Downloads			
.n	Name	Size	Modify time	.n	Name	Size	Modify time
UP--DIR	UP--DIR	Jan 27 12:03		UP--DIR	UP--DIR	Jan 27 12:25	
in_out.asm	3942	Jan 27 13:17		in_out.asm	3942	Jan 27 13:17	
*lab5-1	8788	Jan 27 12:28		lab01_report.pdf	852586	Jan 25 14:07	
lab5-1.asm	1890	Jan 27 12:20		НКАбд-02-25 Ром-ексей 1 ЛР.pdf	852586	Jan 25 14:07	
lab5-1.o	752	Jan 27 12:28		НКАбд-02-25 Ром-ексей 2 ЛР.docx	577730	Jan 26 18:09	
				НКАбд-02-25 Ром-лексей 2 ЛР.md	36900	Jan 26 18:19	
				НКАбд-02-25 Ром-ексей 2 ЛР.pdf	724926	Jan 26 16:48	
1,890 B in 1 file				in_out.asm			
UP--DIR				in_out.asm			
1775M / 24G (7%)				1775M / 24G (7%)			

Далее выберите файл lab5-1.asm в папке lab05, нажмите F6 (Переименовать/Переместить) и введите новое имя lab5-2.asm, чтобы создать копию.



Откройте файл `lab5-2.asm` для редактирования (F4) и измените код в соответствии с листингом 5.2, используя директиву `%include` и операторы вызова.

```

GNU nano 8.4 /home/fernando/work/arch-pc/lab0
; ---
; Программа вывода сообщения на экран и ввода строки с клавиатуры
; ---
; --- Объявление переменных ---
SECTION .data ; Секция инициализированных данных
msg: DB 'Введите строку:',10 ; сообщение плюс символ перевода строки
msgLen: EQU $-msg ; Длина переменной 'msg'
SECTION .bss ; Секция не инициализированных данных
buf1: RESB 80 ; Буфер размером 80 байт
; --- Текст программы ---
SECTION .text ; Код программы
GLOBAL start ; Начало программы
start: ; Точка входа в программу
; --- Системный вызов write
mov eax,4 ; Системный вызов для записи (sys_write)
mov ebx,1 ; Описатель файла 1 - стандартный вывод
mov ecx,msg ; Адрес строки 'msg' в 'ecx'
mov edx,msgLen ; Размер строки 'msg' в 'edx'
int 80h ; Вызов ядра
; --- Системный вызов read ---
mov eax,3 ; Системный вызов для чтения (sys_read)
mov ebx,0 ; Дескриптор файла 0 - стандартный вывод
mov ecx,buf1 ; Адрес буфера под вводимую строку
mov edx,80 ; Длина вводимой строки
int 80h ; Вызов ядра
; --- Системный вызов exit ---
mov eax,1 ; Системный вызов для выхода (sys_exit)
mov ebx,0 ; Выход с кодом возврата 0 (без ошибок)
int 80h ; Вызов ядра
%include 'in_out.asm'

SECTION .data
msg db 'Введите строку:',0

SECTION .bss
buf1 resb 80

```

Сохраните файл и закройте редактор. Снова скройте панели (Ctrl + O), скомпилируйте и запустите lab5-2.asm.

```

fernando@ubuntu25-10:~/work/arch-pc/lab05$ nasm -f elf lab5-2.asm
fernando@ubuntu25-10:~/work/arch-pc/lab05$ ld -m elf_i386 -o lab5-2 lab5-2.o
fernando@ubuntu25-10:~/work/arch-pc/lab05$ ./lab5-2
Введите строку:
Fernando Alexei

```

Программа должна работать так же, как lab5-1.asm.

Откройте lab5-2.asm снова и замените "call sprintLF" на "call sprint". Сохраните, скомпилируйте и запустите.

```
fernando@ubuntu25-10:~/work/arch-pc/lab05$ nano lab5-2.asm
fernando@ubuntu25-10:~/work/arch-pc/lab05$ nasm -f elf lab5-2.asm
fernando@ubuntu25-10:~/work/arch-pc/lab05$ ld -m elf_i386 -o lab5-2 lab5-2.o
fernando@ubuntu25-10:~/work/arch-pc/lab05$ ./lab5-2
Введите строку:
Fernando Alexei
```

Выполнение самостоятельной работы

Задание 1

1. Создайте копию файла lab5-1.asm. Внесите изменения в программу (без использования внешнего файла in_out.asm), так чтобы она работала по следующему алгоритму:

- вывести приглашение типа “Введите строку:”;
- ввести строку с клавиатуры;
- вывести введенную строку на экран.

```
GNU nano 8.4 lab5-1-modificado.asm *
SECTION .data
    msg: DB 'Введите строку:',10      ; сообщение + перевод строки
    msglen: EQU $-msg                ; Длина переменной 'msg'

SECTION .bss
    buf1: RESB 80                    ; Буфер размером 80 байт

SECTION .text
    GLOBAL _start

_start:
    ; 1. Системный вызов write - вывод приглашения
    mov eax, 4                      ; sys_write
    mov ebx, 1                      ; стандартный вывод
    mov ecx, msg                    ; адрес строки
    mov edx, msglen                 ; длина строки
    int 80h

    ; 2. Системный вызов read - чтение строки
    mov eax, 3                      ; sys_read
    mov ebx, 0                      ; стандартный ввод
    mov ecx, buf1                   ; адрес буфера
    mov edx, 80                     ; максимальная длина
    int 80h

    ; ===== НОВЫЙ КОД - НАЧАЛО =====
    ; Сохраняем длину введенной строки
    mov esi, eax                    ; sys_read возвращает длину в eax

    ; 3. Системный вызов write - вывод введенной строки
    mov eax, 4                      ; sys_write
    mov ebx, 1                      ; стандартный вывод
    mov ecx, buf1                   ; адрес буфера (где хранится введенная строка)
    mov edx, esi                    ; длина введенной строки
    int 80h

    ; ===== НОВЫЙ КОД - КОНЕЦ =====
```

Задание 2

2. Получите исполняемый файл и проверьте его работу. На приглашение ввести строку введите свою фамилию.

```
fernando@ubuntu25-10:~/work/arch-pc/lab05$ nasm -f elf lab5-1_mod.asm
fernando@ubuntu25-10:~/work/arch-pc/lab05$ ld -m elf_i386 -o lab5-1_mod lab5-1_mod.o
fernando@ubuntu25-10:~/work/arch-pc/lab05$ ./lab5-1_mod
Введите строку:
Fernando Alexei
Fernando Alexei
```

Задание 3

3. Создайте копию файла lab5-2.asm. Исправьте текст программы с использованием подпрограмм из внешнего файла in_out.asm, так чтобы она работала по следующему алгоритму: • вывести приглашение типа “Введите строку:”; • ввести строку с клавиатуры; • вывести введенную строку на экран

```
GNU nano 8.4 lab5-2-mod.asm *
#include 'in_out.asm' ; подключение внешнего файла

SECTION .data
    msg: DB 'Введите строку: ',0h ; Приглашение

SECTION .bss
    buf1: RESB 80 ; Буфер для вводимой строки

SECTION .text
GLOBAL _start
_start:
    ; 1. Выводим приглашение
    mov eax, msg
    call sprintLF ; Используем sprintLF (не printfLF)

    ; 2. Читаем ввод в buf1
    ; sread ожидает адрес в ECX и длину в EDX
    mov ecx, buf1
    mov edx, 80
    call sread

    ; 3. Выводим содержимое buf1 на экран
    mov eax, buf1 ; sprintLF ожидает адрес в EAX
    call sprintLF ; Печатаем то, что в буфере (с переводом строки)

    ; 4. Выход
    call quit

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```

```
fernando@ubuntu25-10:~/work/arch-pc/lab05$ nasm -f elf lab5-2-mod.asm
fernando@ubuntu25-10:~/work/arch-pc/lab05$ ld -m elf_i386 -o lab5-2-mod lab5-2-mod.o
fernando@ubuntu25-10:~/work/arch-pc/lab05$ ./lab5-2-mod
Введите строку:
Fernando Alexei
Fernando Alexei
```

Выводы

Вы приобретете практические навыки работы с файловым менеджером Midnight Commander: научиться создавать каталоги (F7), файлы, редактировать их (F4), просматривать их (F3), копировать их (F5) и переименовывать (F6). Вы также освоите базовую структуру программы на ассемблере NASM (разделы `.data`, `.bss`, `.text`) и научитесь использовать следующие ключевые инструкции:

- `mov`: для перемещения данных (адресов, констант) в регистры (`eax`, `ebx`, `ecx`, `edx`) для подготовки системных вызовов.

- `int 80h`: для вызова ядра Linux и выполнения системных функций, таких как `sys_write`, `sys_read` и `sys_exit`.

Вы также научитесь включать внешние файлы (`%include`) и использовать подпрограммы (`call`), что значительно упрощает программирование.

Список литературы

1. Debian Administration. (2023). An introduction to Midnight Commander. Получено с https://debian-administration.org/article/An_introduction_to_the_Midnight_Commander_file_manager
2. Die.net. (2023). Руководство по mc (страница man). Получено с <https://linux.die.net/man/1/mc>
3. Linux Documentation Project. (2022). Руководство по Midnight Commander. Получено с <https://tldp.org/LDP/GNU-Linux-Tools-Summary/html/mc.html>
4. Midnight Commander Development Team. (2023). Официальная документация. Получено с <https://midnight-commander.org/wiki/doc>
5. OpenSource.com. (2022). Использование Midnight Commander для управления файлами в Linux. Получено с <https://opensource.com/article/19/8/using-midnight-commander>
6. Red Hat Enterprise Linux Documentation. (2023). Использование файлового менеджера Midnight Commander. Получено с https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/8/html/using_the_desktop_environment_in_rhel_8/using-midnight-commander_file-manager_using-the-desktop-environment-in-rhel-8
7. Intel Corporation. (2023). *Intel 64 and IA-32 Architectures Software Developer Manuals*. Retrieved from <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
8. GNU Project. (2023). *Binutils Documentation*. Retrieved from <https://sourceware.org/binutils/docs-2.40>